

Statistical Validation

Business Analyst Case Study | Dataset: Olist E-Commerce (seller marketplace)

04 · STATISTICAL VALIDATION

Why Statistical Validation Was Needed

The seller-tenure finding (1.08% vs. 0.40% cancellation rate) is a meaningful-looking gap, but a gap of this size could, in principle, occur by random chance alone, particularly for the smaller-volume buckets. A chi-square test of independence was used to check whether the relationship between seller tenure and cancellation outcome is statistically real, or consistent with random noise.

The Underlying Logic, in Plain Terms

The test compares two versions of the same table: what was **observed** in the data, and what we would **expect** to see if seller tenure had no relationship to cancellation at all (i.e., if every bucket cancelled at the same overall baseline rate). The larger the gap between observed and expected counts, the less plausible it is that the pattern arose by chance.

Step 1 — The Observed Data

This test operates on the order-item level table (112,650 rows total), since seller tenure is a seller-level attribute and one order can span multiple sellers.

Seller Tenure Bucket	Total Rows	Problem Rows	Cancellation Rate
0-30 days (new)	12,253	132	1.08%
30-90 days	16,451	85	0.52%
90+ days (established)	83,946	332	0.40%
Total	112,650	549	0.49% (baseline)

Step 2 — The Expected Data ("If Tenure Didn't Matter")

The baseline rate across all rows is $549 \div 112,650 = 0.4873\%$. If tenure had zero effect, every bucket should show approximately this same rate. Expected problem-row counts are calculated as: bucket size $\times$ baseline rate.

Bucket	Bucket Size	Expected Problem Rows (Size $\times$ 0.4873%)	Actual Problem Rows	Gap
0-30 days (new)	12,253	≈ 59.7	132	+72.3
30-90 days	16,451	≈ 80.2	85	+4.8
90+ days (established)	83,946	≈ 409.1	332	-77.1

The new-seller bucket shows more than double the expected number of problem rows. The established-seller bucket shows notably fewer than expected. Both gaps point in the same direction as the hypothesis — this consistency across all three buckets is what the chi-square statistic captures mathematically.

Step 3 — Computing the Chi-Square Statistic

For each cell in the table (each bucket × each outcome, problem/non-problem), the test computes:

$$(\text{Observed} - \text{Expected})^2 \div \text{Expected}$$

...and sums this value across every cell in the table (both the 'problem' and 'non-problem' columns for all three buckets, six cells total). This produces a single number summarizing the total mismatch between what was observed and what "no effect" would predict.

Result: Chi-square statistic = 102.83, with 2 degrees of freedom (calculated as $(\text{rows} - 1) \times (\text{columns} - 1) = (3 - 1) \times (2 - 1) = 2$, since there are 3 tenure buckets and 2 outcome categories — problem / non-problem).

Step 4 — From Chi-Square Statistic to p-value

The chi-square statistic (102.83) is then converted into a p-value using the chi-square distribution for 2 degrees of freedom. The p-value directly answers the practical question: "if seller tenure truly had no effect, what is the probability of seeing a mismatch this large purely by chance?"

Result: p-value < 0.001 (well under one-tenth of one percent).

Step 5 — Interpreting the Result

The conventional significance threshold in statistics is $p < 0.05$ (a 5% chance of the pattern being coincidental). This result ($p < 0.001$) clears that threshold by more than an order of magnitude — it is not a borderline or marginal result. This upgrades the seller-tenure finding from an observed pattern to a statistically validated driver of cancellation risk.

Reference: Python Implementation

```
from scipy.stats import chi2_contingency
import numpy as np

data = {
    '0-30 days (new)': {'total': 12253, 'problem': 132},
    '30-90 days': {'total': 16451, 'problem': 85},
    '90+ days (established)': {'total': 83946, 'problem': 332},
}

contingency_table = []
for bucket, vals in data.items():
    non_problem = vals['total'] - vals['problem']
    contingency_table.append([vals['problem'], non_problem])

chi2, p, dof, expected = chi2_contingency(np.array(contingency_table))
# chi2 = 102.83, p < 0.001, dof = 2
```